Lab Exercises 5-7

5.Single Inheritance
A company wants to manage employee salary details. All employees have common details such as employee ID, name, and basic salary. Permanent employees receive an additional HRA and DA.

Tasks
Create a superclass Employee.
Create a subclass PermanentEmployee.
Calculate the gross salary.
Display all employee details.

Program:

```
class Employee {
    int employeeId;
    String name;
    double basicSalary;


    Employee(int employeeId, String name, double basicSalary) {
        this.employeeId = employeeId;
        this.name = name;
        this.basicSalary = basicSalary;
    }

     void displayDetails() {
        System.out.println("Employee ID   : " + employeeId);
        System.out.println("Name          : " + name);
        System.out.println("Basic Salary  : " + basicSalary);
    }
}

class PermanentEmployee extends Employee {
    double hra;
    double da;
```

```
    PermanentEmployee(int employeeId, String name, double basicSalary,
                      double hra, double da) {
        super(employeeId, name, basicSalary);
        this.hra = hra;
        this.da = da;
    }

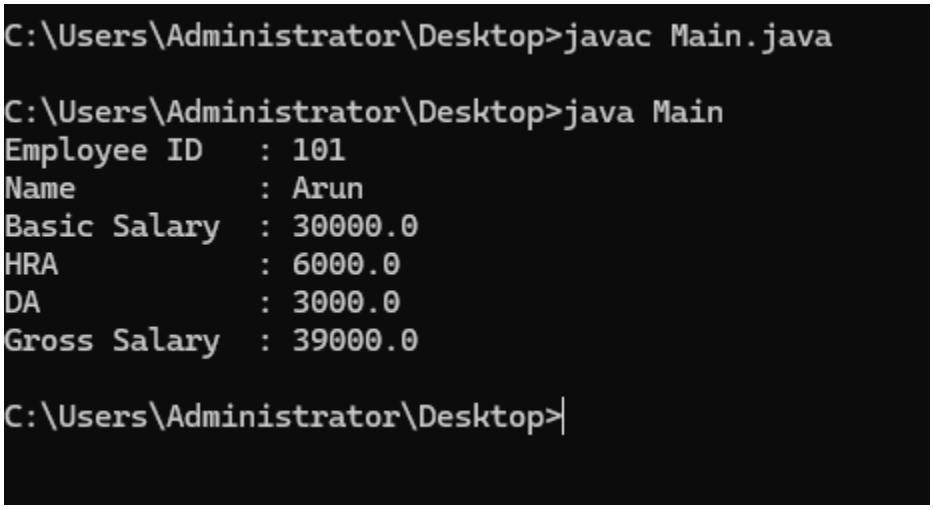
       double calculateGrossSalary() {
        return basicSalary + hra + da;
    }

      @Override
    void displayDetails() {
        super.displayDetails();
        System.out.println("HRA           : " + hra);
        System.out.println("DA            : " + da);
        System.out.println("Gross Salary  : " + calculateGrossSalary());
    }
}

public class Main {
    public static void main(String[] args) {
        PermanentEmployee emp = new PermanentEmployee(
            101, "Arun", 30000, 6000, 3000
        );

        emp.displayDetails();
    }
}
```

Output:



6)Multilevel Inheritance

A university wants to automate student result processing.

Student stores student information.
Marks stores marks in five subjects.
Result calculates total, average, and grade.

Program:

```
class Student {
    String name;
    int rollNo;

    Student(String name, int rollNo) {
        this.name = name;
        this.rollNo = rollNo;
    }

    void displayStudentDetails() {
```

```
        System.out.println("Name    : " + name);
        System.out.println("Roll No : " + rollNo);
    }
}

class Marks extends Student {
    int mark1, mark2, mark3, mark4, mark5;

    Marks(String name, int rollNo, int mark1, int mark2,
          int mark3, int mark4, int mark5) {

        super(name, rollNo);

        this.mark1 = mark1;
        this.mark2 = mark2;
        this.mark3 = mark3;
        this.mark4 = mark4;
        this.mark5 = mark5;
    }

    void displayMarks() {
        System.out.println("Subject 1 : " + mark1);
        System.out.println("Subject 2 : " + mark2);
        System.out.println("Subject 3 : " + mark3);
        System.out.println("Subject 4 : " + mark4);
        System.out.println("Subject 5 : " + mark5);
    }
}

class Result extends Marks {

    Result(String name, int rollNo, int mark1, int mark2,
           int mark3, int mark4, int mark5) {

        super(name, rollNo, mark1, mark2, mark3, mark4, mark5);
    }

    void calculateResult() {
        int total = mark1 + mark2 + mark3 + mark4 + mark5;
        double average = total / 5.0;

        char grade;

        if (average >= 90)
```

```
            grade = 'A';
        else if (average >= 80)
            grade = 'B';
        else if (average >= 70)
            grade = 'C';
        else if (average >= 60)
            grade = 'D';
        else if (average >= 50)
            grade = 'E';
        else
            grade = 'F';

        System.out.println("Total   : " + total);
        System.out.println("Average : " + average);
        System.out.println("Grade   : " + grade);
    }

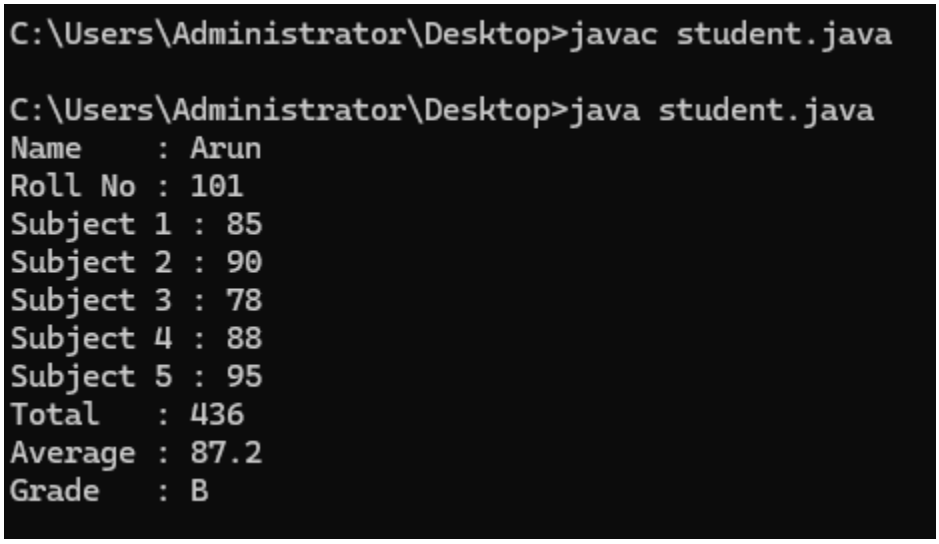
    void displayResult() {
        displayStudentDetails();
        displayMarks();
        calculateResult();
    }
}

public class student {
    public static void main(String[] args) {

        Result student = new Result(
            "Arun", 101, 85, 90, 78, 88, 95
        );

        student.displayResult();
    }
}
```

Output:



7)Multiple Inheritance Through Interfaces

Program:

```
interface Academic {
    void displayAcademicDetails();
}

interface Sports {
    void displaySportsDetails();
}

class Student implements Academic, Sports {
    String name;
    int rollNo;
    int marks;
    String sport;

    Student(String name, int rollNo, int marks, String sport) {
        this.name = name;
        this.rollNo = rollNo;
        this.marks = marks;
```

```
        this.sport = sport;
    }

    public void displayAcademicDetails() {
        System.out.println("Name   : " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Marks  : " + marks);
    }

    public void displaySportsDetails() {
        System.out.println("Sport  : " + sport);
    }
}

public class academic {
    public static void main(String[] args) {
        Student student = new Student(
            "Arun", 101, 85, "Cricket"
        );

        student.displayAcademicDetails();
        student.displaySportsDetails();
    }
}
```

Output:

```
C:\Users\Administrator\Desktop>javac academic.java

C:\Users\Administrator\Desktop>java academic.java
Name   : Arun
Roll No: 101
Marks  : 85
Sport  : Cricket
```